

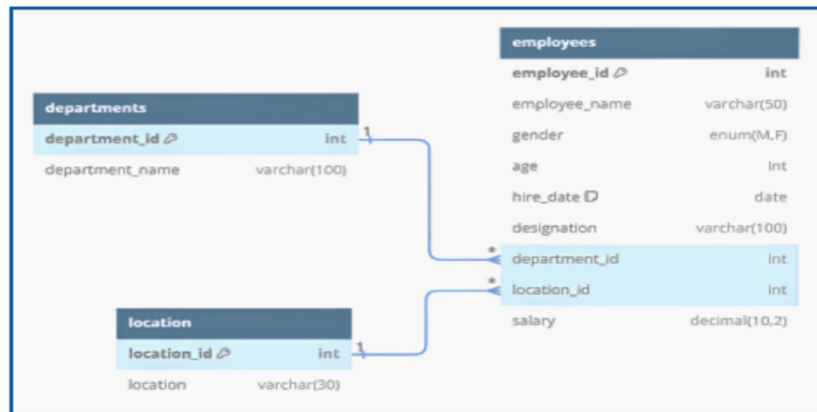
Data Analytics (DA)

Module -3

MySQL Assignment 2 - Querying Data

Task

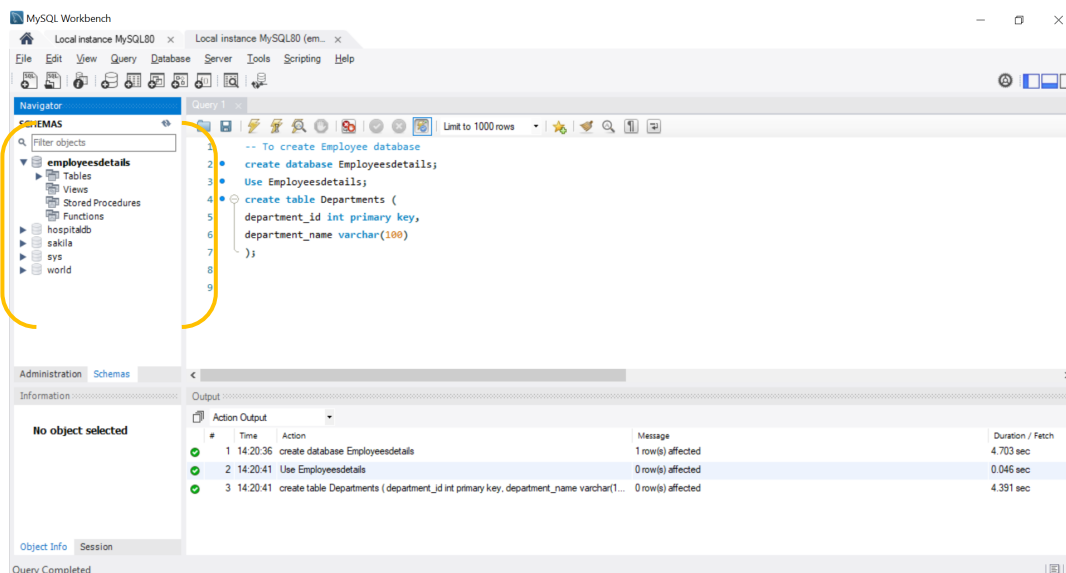
Dataset Description:



• Table: Departments

➤ Attributes:

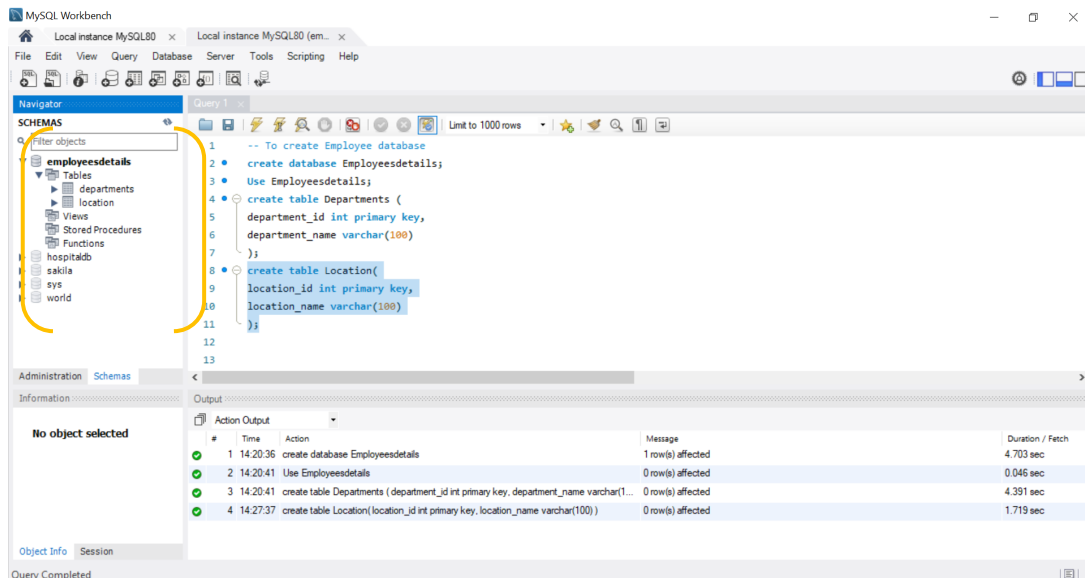
1. department_id – Primary Key
2. department_name



• Table: Location

➤ Attributes:

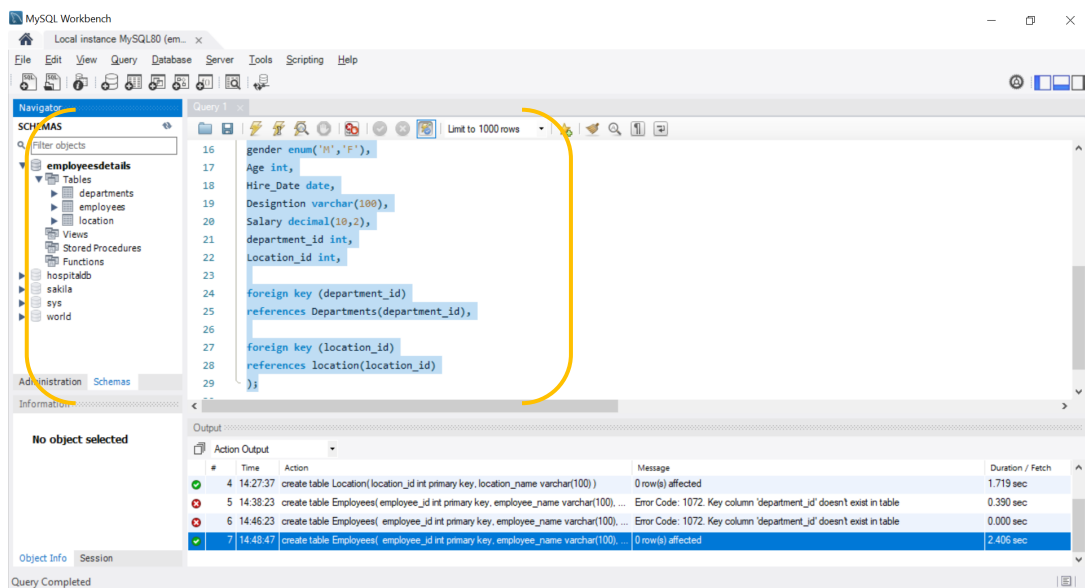
1. location_id – Primary Key
2. location_name



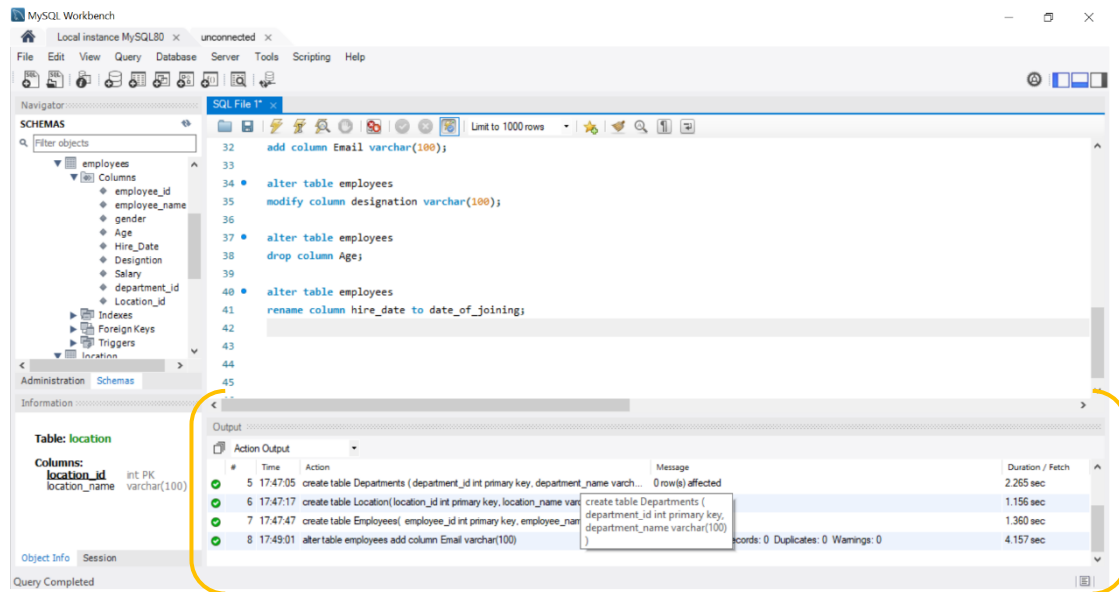
• Table: Employees

➤ Attributes:

1. employee_id – Primary Key
2. Employee_name
3. Gender
4. Age
5. Hire_date
6. Designation
7. Salary
8. department_id – Foreign Key referencing department(department_id)
9. location_id—Foreign Key referencing location(location_id)



OUTPUT:

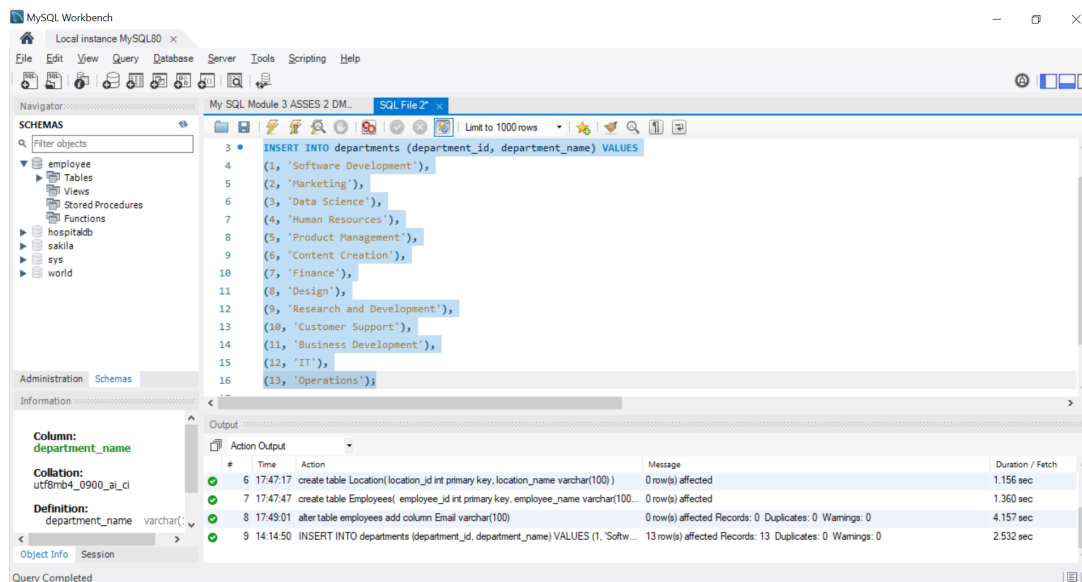


Q:Problem Statement:

As a Data Analyst, you will continue working with the Employee Database created in the previous assignment. The database already contains the required tables, relationships, and constraints.

Use this DML insert code, **DML Insertion- Employee Data**, to populate the tables

Insert departments Data:



Inserted Data OUTPUT:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'departments' table. The main editor window contains the following SQL code:

```
12 (9, 'Research and Development'),
13 (10, 'Customer Support'),
14 (11, 'Business Development'),
15 (12, 'IT'),
16 (13, 'Operations');
17
18 SELECT * FROM Departments;
19
```

The 'Result Grid' shows the output of the SELECT statement:

department_id	department_name
1	Software Development
2	Marketing
3	Data Science
4	Human Resources
5	Product Management

The 'Output' pane at the bottom shows the execution results of the INSERT statement:

#	Time	Action	Message	Duration / Fetch
9	14:14:50	INSERT INTO departments (department_id, department_name) VALUES (1, 'Soft...	13 row(s) affected Records: 13 Duplicates: 0 Warnings: 0	2.532 sec
10	14:20:02	USE Employeesdetails	0 row(s) affected	0.312 sec
11	14:20:03	INSERT INTO departments (department_id, department_name) VALUES (1, 'Soft...	Error Code: 1062 Duplicate entry '1' for key 'departments.PRIMARY'	0.766 sec
12	14:20:19	SELECT * FROM Departments LIMIT 0 1000	13 row(s) returned	0.047 sec / 0.000 sec

Insert Location Data:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the database schema, including the 'location' table. The main editor window contains the following SQL code:

```
12 (9, 'Research and Development'),
13 (10, 'Customer Support'),
14 (11, 'Business Development'),
15 (12, 'IT'),
16 (13, 'Operations');
17
18 SELECT * FROM Departments;
19
20 INSERT INTO location (location_id, location_name) VALUES
21 (1, 'Chennai'),
22 (2, 'Bangalore'),
23 (3, 'Hyderabad'),
24 (4, 'Pune');
25
26
```

The 'Output' pane at the bottom shows the execution results of the INSERT statement:

#	Time	Action	Message	Duration / Fetch
15	14:34:36	DESCRIBE location	2 row(s) returned	0.438 sec / 0.000 sec
16	14:36:19	INSERT INTO location (location_name) VALUES (Chennai), (Bangalore), (Hyde...	Error Code: 1364 Field 'location_id' doesn't have a default value	0.109 sec
17	14:45:30	INSERT INTO location (location_id, location_name) VALUES (1, Chennai), (2, Ba...	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.484 sec

Inserted Location Data OUTPUT:

The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the 'location' table selected. The main editor shows the following SQL queries:

```
20 INSERT INTO location (location_id, location_name) VALUES
21 (1,'Chennai'),
22 (2,'Bangalore'),
23 (3,'Hyderabad'),
24 (4,'Pune');
25
26 SELECT * FROM location;
```

The 'Result Grid' shows the output of the SELECT query:

location_id	location_name
1	Chennai
2	Bangalore
3	Hyderabad
4	Pune

The 'Output' pane shows the execution results of the queries:

#	Time	Action	Message	Duration / Fetch
16	14:36:19	INSERT INTO location (location_name) VALUES ('Chennai'), ('Bangalore'), ('Hyderabad'), ('Pune');	Error Code: 1364. Field 'location_id' doesn't have a default value	0.109 sec
17	14:45:30	INSERT INTO location (location_id, location_name) VALUES (1,'Chennai'), (2,'Bangalore'), (3,'Hyderabad'), (4,'Pune');	4 row(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.484 sec
18	14:47:31	SELECT * FROM location LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec

Insert Location Datas:

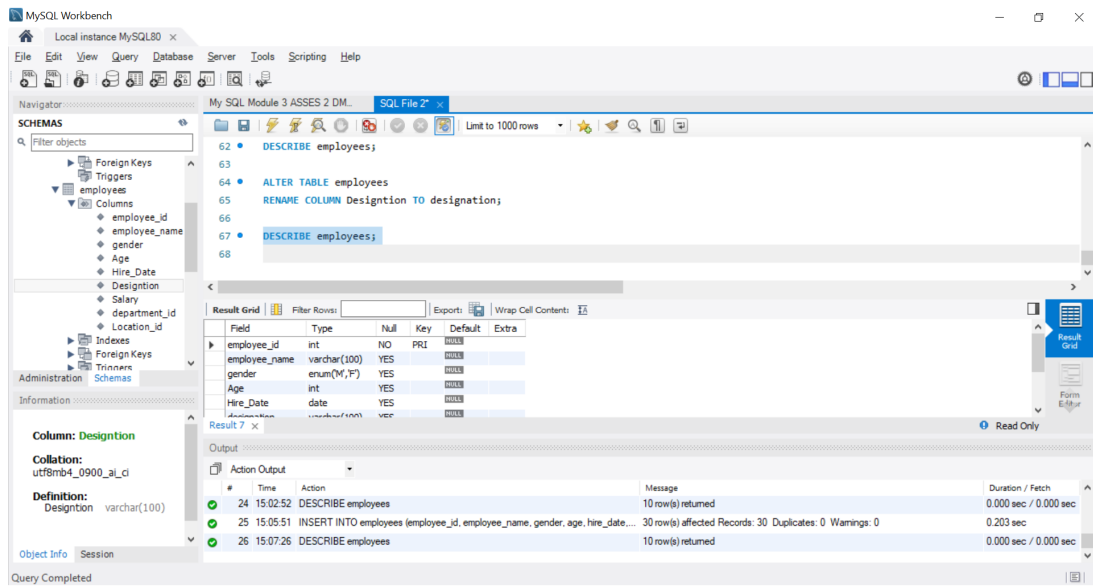
The screenshot shows the MySQL Workbench interface. The left sidebar displays the 'SCHEMAS' tree with the 'employees' table selected. The main editor shows the following SQL queries:

```
42 ('5014', 'Vihaan Mohan', 'M', 30, '2017-03-10', 'Supply Chain Analyst', 10, 2, 60000),
43 ('5015', 'Ishaan Kumar', 'M', 27, '2017-05-20', 'Financial Analyst', 7, 1, 85000),
44 ('5016', 'Zoya Khan', 'F', 31, '2017-07-05', 'Legal Counsel', 4, 4, 100000),
45 ('5017', 'Kabir Nair', 'M', 28, '2017-09-11', 'IT Support Specialist', 12, 2, 80000),
46 ('5018', 'Ishan Mishra', 'M', 25, '2017-11-20', 'Research Scientist', 9, 3, 75000),
47 ('5019', 'Ishika Patel', 'F', 29, '2018-01-15', 'Talent Acquisition Specialist', 4, 4, 55000),
48 ('5020', 'Aarav Nair', 'M', 32, '2018-03-05', 'Software Engineer', 1, 1, 90000),
49 ('5021', 'Advik Kapoor', 'M', 26, '2018-05-20', 'Finance Analyst', 7, 3, 85000),
50 ('5022', 'Aadhya Iyengar', 'F', 28, '2018-07-10', 'HR Specialist', 4, 4, 60000),
51 ('5023', 'Anika Paul', 'F', 30, '2018-09-20', 'Public Relations Specialist', 2, 2, 70000),
52 ('5024', 'Aryan Shetty', 'M', 27, '2018-11-05', 'Product Manager', 5, 1, 95000),
53 ('5025', 'Avni Iyengar', 'F', 31, '2019-01-20', 'Data Scientist', 3, 4, 100000),
54 ('5026', 'Vivaan Singh', 'M', 29, '2019-03-10', 'Business Analyst', 3, 2, 75000),
55 ('5027', 'Ananya Paul', 'F', 32, '2019-05-20', 'Content Writer', 6, 3, 60000),
56 ('5028', 'Anaya Kapoor', 'F', 26, '2019-07-05', 'Event Coordinator', 6, 1, 60000),
57 ('5029', 'Arun Kumar', 'M', 33, '2019-09-11', 'Quality Assurance Analyst', 12, 2, 80000);
```

The 'Output' pane shows the execution results of the queries:

#	Time	Action	Message	Duration / Fetch
23	15:01:48	ALTER TABLE employees RENAME COLUMN Designation TO designation	0 row(s) affected Records: 0 Duplicates: 0 Warnings: 0	2.609 sec
24	15:02:52	DESCRIBE employees	10 row(s) returned	0.000 sec / 0.000 sec
25	15:05:51	INSERT INTO employees (employee_id, employee_name, gender, age, hire_date, designation, salary, department_id, location_id)	30 row(s) affected Records: 30 Duplicates: 0 Warnings: 0	0.203 sec

Inserted Employee Data OUTPUT:



Tasks

Clause & Operators:

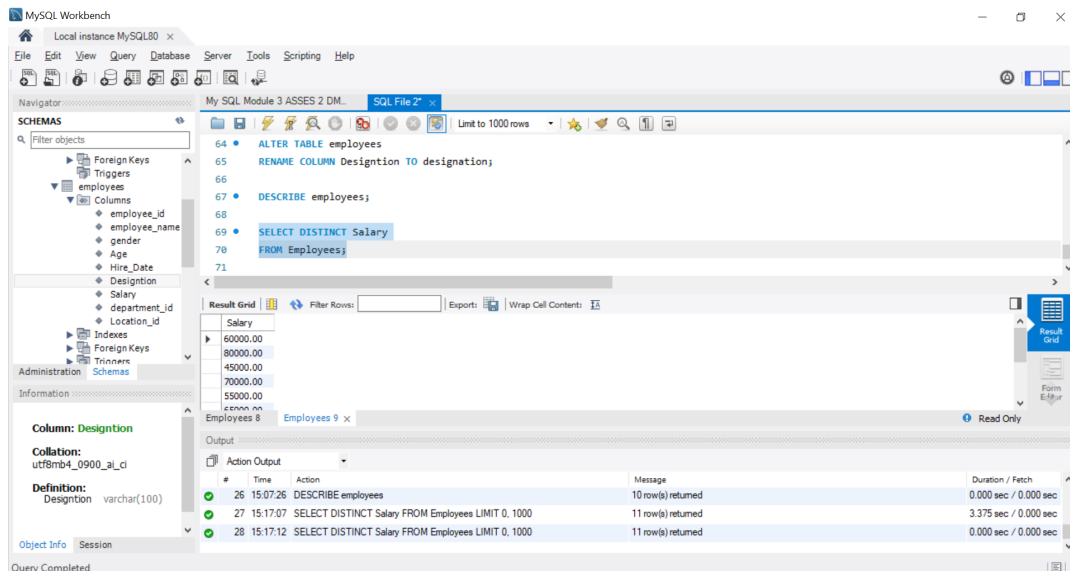
1. DISTINCT VALUES:

- A query to retrieve distinct salaries from the Employees table.

Syntax:

SELECT DISTINCT Salary
FROM Employees;

OUTPUT:



2. ALIAS (AS):

- Provide aliases for the "age" and "salary" columns as "Employee_Age" and "Employee_Salary", respectively.

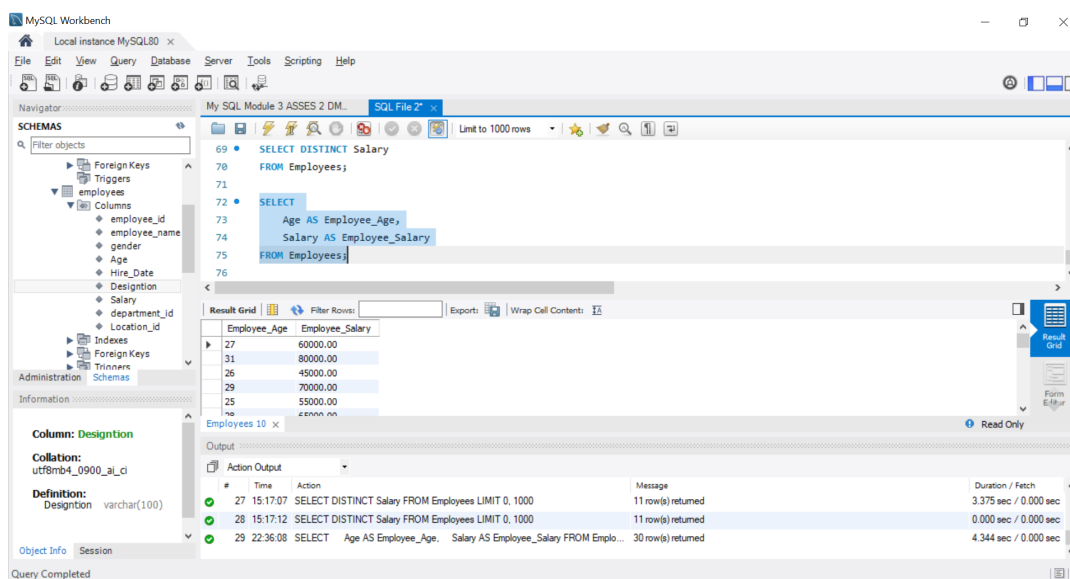
Syntax:

SELECT

Age AS Employee_Age,
Salary AS Employee_Salary

FROM Employees;

OUTPUT:



3. WHERE CLAUSE & OPERATORS:

- Retrieve employees with a salary greater than ₹50000 and hired before 2016-01-01.

✓ Use the WHERE clause with both conditions:

Syntax:

```
SELECT *  
FROM Employees  
WHERE Salary > 50000  
AND Hire_Date < '2016-01-01';
```

OUTPUT

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
SELECT *  
FROM Employees  
WHERE Salary > 50000  
AND Hire_Date < '2016-01-01';
```

The Results Grid displays the following data:

employee_id	employee_name	gender	Age	Hire_Date	designation	Salary	department_id	Location_id	Email
5001	Vihann Singh	M	27	2015-01-20	Data Analyst	60000.00	3	4	NULL
5002	Reyansh Singh	M	31	2015-03-10	Network Engineer	80000.00	12	1	NULL
5004	Kiara Malhotra	F	29	2015-07-05	NULL	70000.00	8	3	NULL
5005	Anvi Chaudhary	F	25	2015-09-11	Business Development Executive	55000.00	11	1	NULL
5006	Dhruv Shetty	M	28	2015-11-20	UI Developer	65000.00	8	2	NULL

The Output tab shows the execution of the query, indicating that 5 rows were returned.

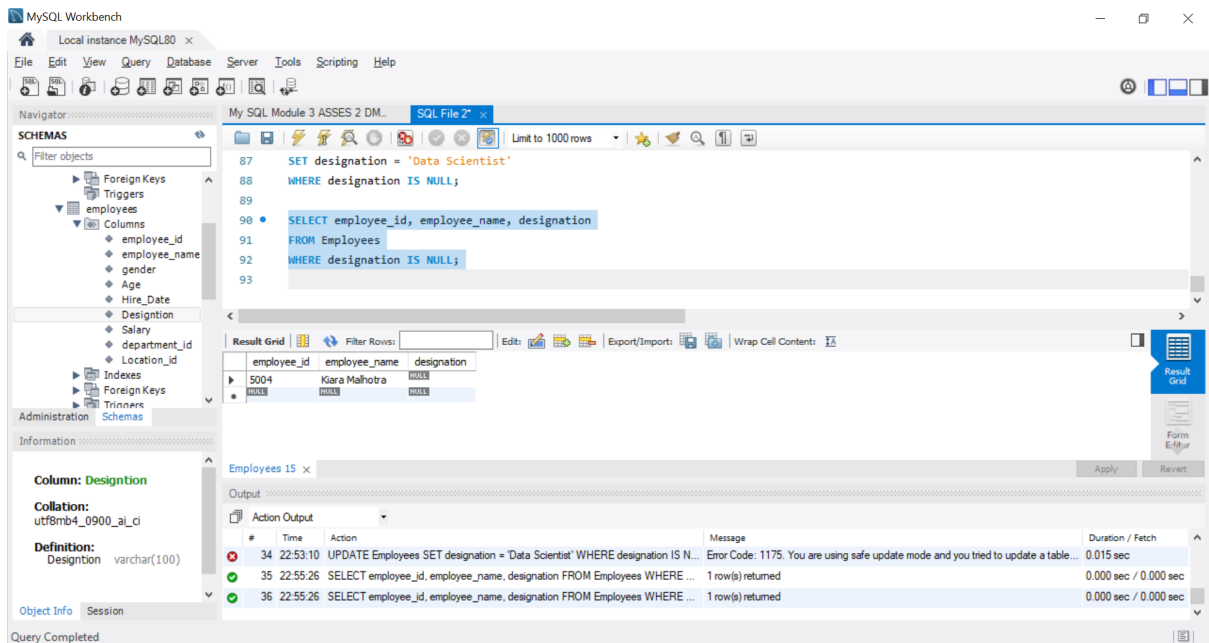
- Find the employee whose designation is missing and fill it with "Data Scientist".

➤ Find the employee with missing designation

Syntax:

```
SELECT employee_id, employee_name, designation  
FROM Employees  
WHERE designation IS NULL;
```

OUTPUT



- **Fill the missing designation with "Data Scientist"**
- **update using the primary key**

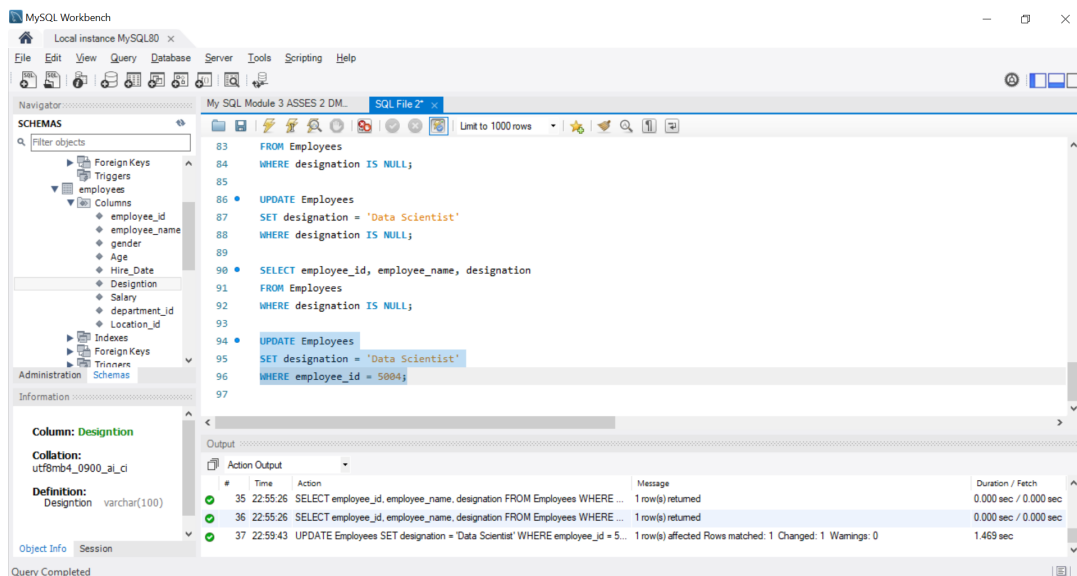
Syntax:

UPDATE Employees

SET designation = 'Data Scientist'

WHERE employee_id = 5004;

Updated:



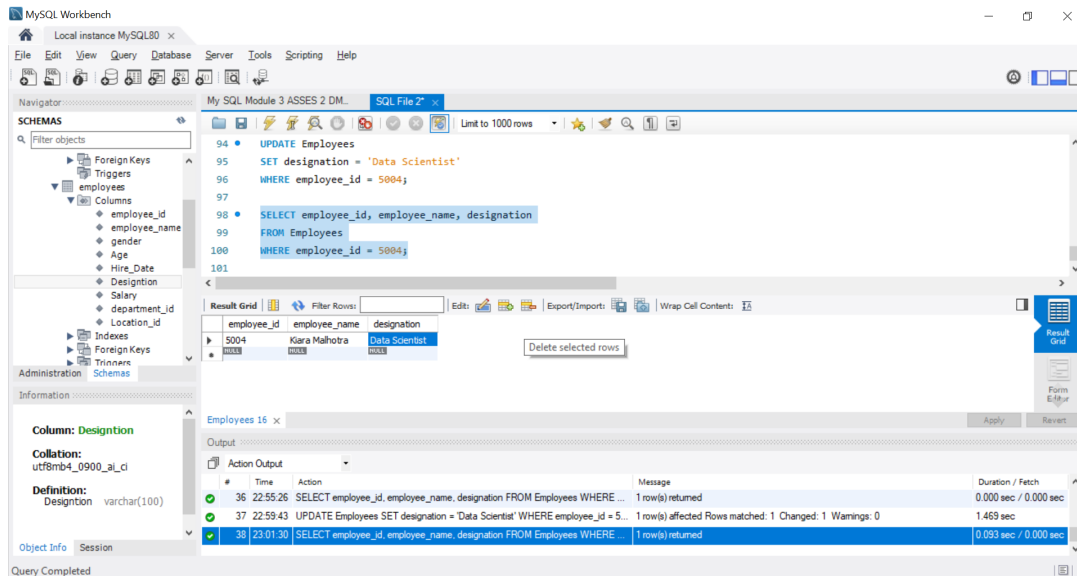
Syntax:

SELECT employee_id, employee_name, designation

FROM Employees

WHERE employee_id = 5004;

OUTPUT



Sorting And Grouping Data:

1. ORDER BY:

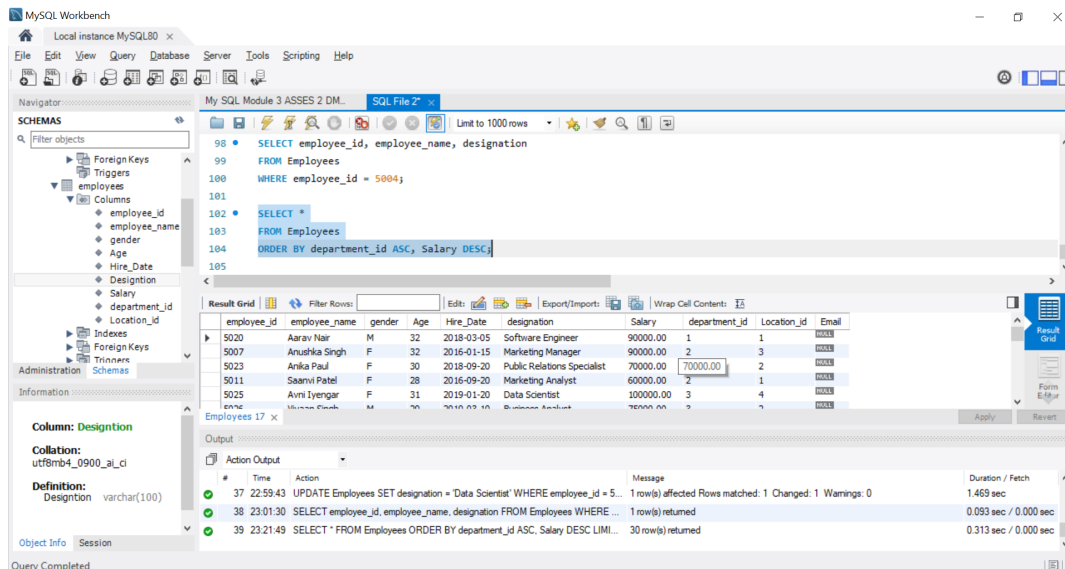
- Find employees sorted by department ID in ascending order and salary in descending order.

✓ Use **ORDER BY** with two columns:

Syntax:

```
SELECT *
FROM Employees
ORDER BY department_id ASC,
Salary DESC;
```

OUTPUT:



2. LIMIT:

- Display the first 5 employees hired in the year 2018.

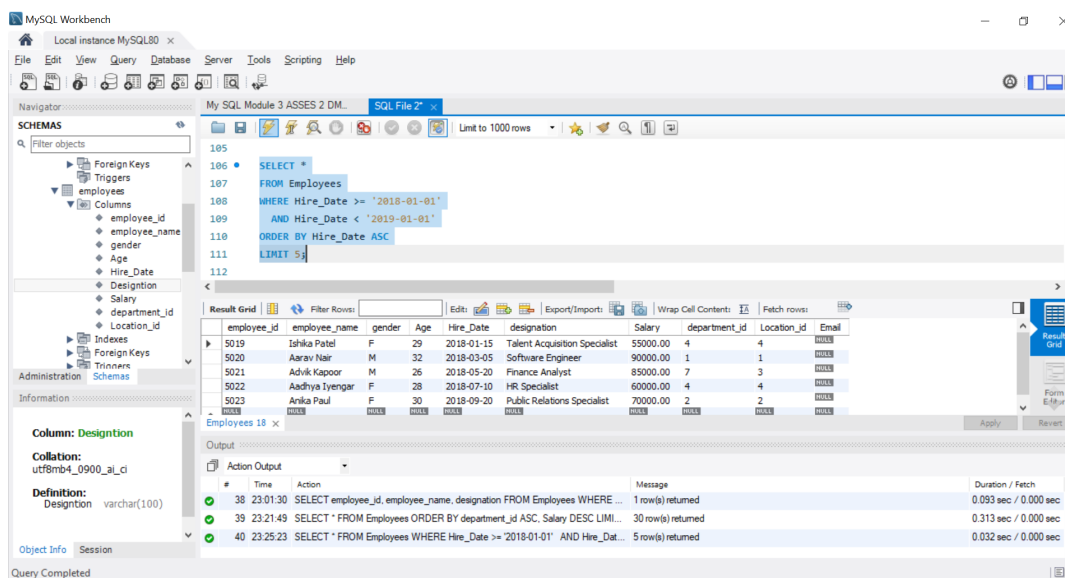
Syntax:

```

SELECT *
FROM Employees
WHERE Hire_Date >= '2018-01-01'
      AND Hire_Date < '2019-01-01'
ORDER BY Hire_Date ASC
LIMIT 5;

```

OUTPUT:



3. AGGREGATE FUNCTIONS:

- Calculate the sum of all salaries in the Finance department.

use the **SUM()** aggregate function.

✓ Departments table contains department_name = 'Finance'

OUTPUT:

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
111 LIMIT 5;
112
113 SELECT SUM(e.Salary) AS Total_Finance_Salary
114 FROM Employees e
115 JOIN Departments d
116 ON e.department_id = d.department_id
117 WHERE d.department_name = 'Finance';
```

The Result Grid shows the output of the query:

Total_Finance_Salary
170000.00

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
39	23:21:49	SELECT * FROM Employees ORDER BY department_id ASC, Salary DESC LIMIT...	30 row(s) returned	0.313 sec / 0.000 sec
40	23:25:23	SELECT * FROM Employees WHERE Hire_Date >= '2018-01-01' AND Hire_Dat...	5 row(s) returned	0.032 sec / 0.000 sec
41	23:29:19	SELECT SUM(e.Salary) AS Total_Finance_Salary FROM Employees e JOIN Dep...	1 row(s) returned	0.344 sec / 0.000 sec

- Find the minimum age among all employees.

Syntax:

SELECT MIN(Age) AS Minimum_Age
FROM Employees;

OUTPUT:

The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL code:

```
114 FROM Employees e
115 JOIN Departments d
116 ON e.department_id = d.department_id
117 WHERE d.department_name = 'Finance';
118
119 SELECT MIN(Age) AS Minimum_Age
120 FROM Employees;
```

The Result Grid shows the output of the query:

Minimum_Age
25

The Output pane shows the execution log:

#	Time	Action	Message	Duration / Fetch
40	23:25:23	SELECT * FROM Employees WHERE Hire_Date >= '2018-01-01' AND Hire_Dat...	5 row(s) returned	0.032 sec / 0.000 sec
41	23:29:19	SELECT SUM(e.Salary) AS Total_Finance_Salary FROM Employees e JOIN Dep...	1 row(s) returned	0.344 sec / 0.000 sec
42	23:32:17	SELECT MIN(Age) AS Minimum_Age FROM Employees LIMIT 0, 1000	1 row(s) returned	0.281 sec / 0.000 sec

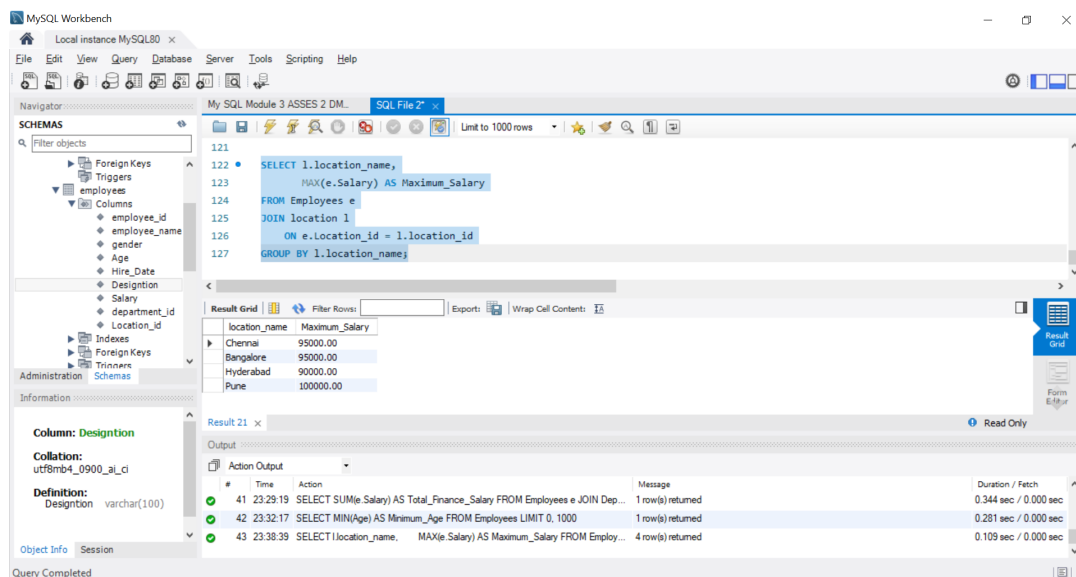
4. GROUP BY:

- List the maximum salary for each location.
 - ✓ Employees table has Location_id, and the location names are stored in the location table:

Syntax:

```
SELECT l.location_name,  
MAX(e.Salary) AS Maximum_Salary  
FROM Employees e  
JOIN location l  
ON e.Location_id = l.location_id  
GROUP BY l.location_name;
```

OUTPUT

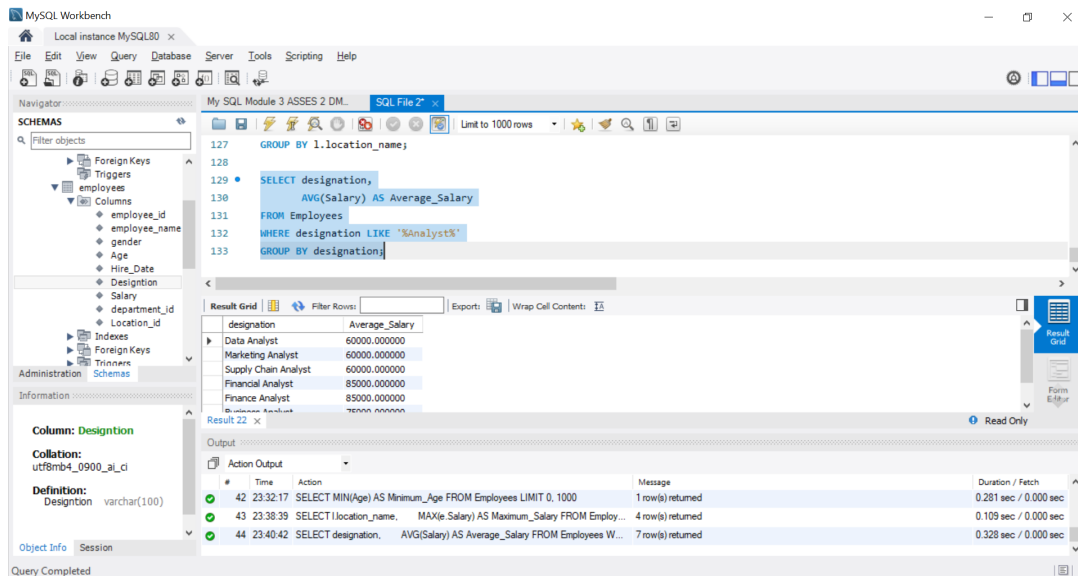


- Calculate the average salary for each designation containing the word 'Analyst'.

- ✓ Use **LIKE** '%Analyst%'

Syntax:

```
SELECT designation,  
AVG(Salary) AS Average_Salary  
FROM Employees  
WHERE designation LIKE '%Analyst%'  
GROUP BY designation;  
OUTPUT:
```



5. HAVING:

- Find departments with less than 3 employees.

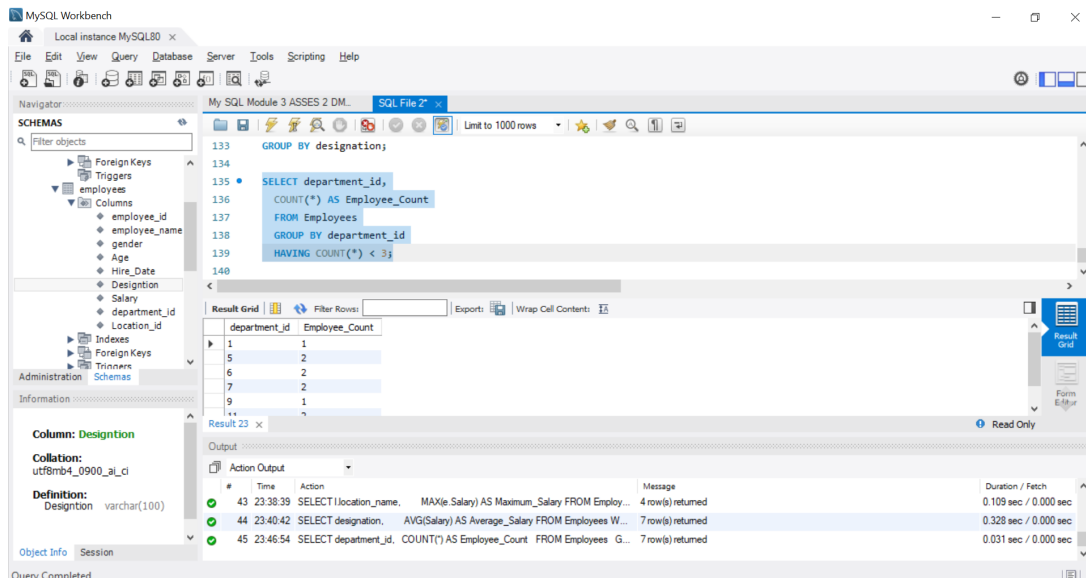
Syntax:

```

SELECT department_id,
COUNT(*) AS Employee_Count
FROM Employees
GROUP BY department_id
HAVING COUNT(*) GROUP BY < 3;

```

OUTPUT:



- Find locations with female employees whose average age is below 30.

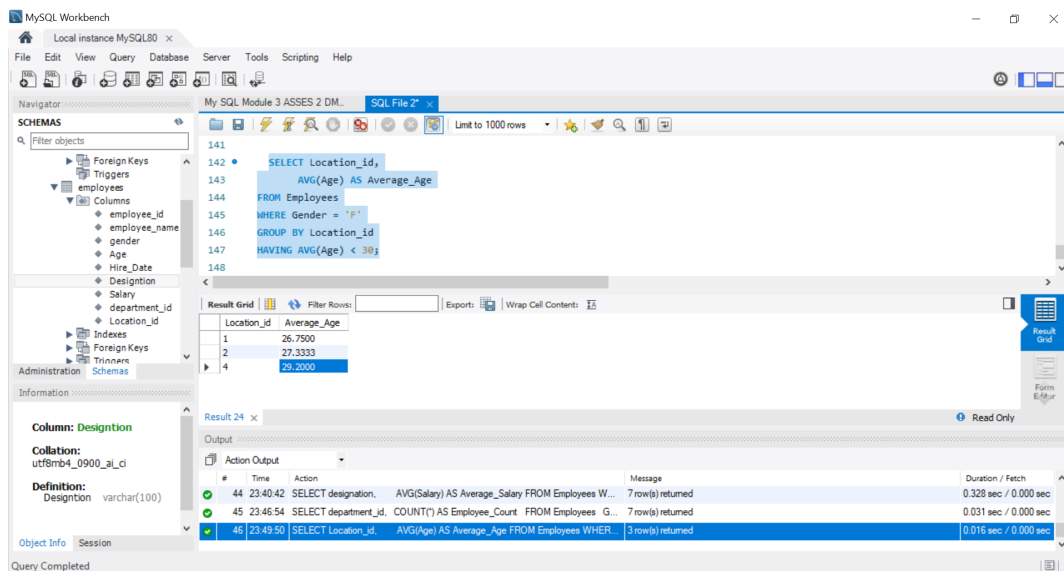
Syntax:

```

SELECT Location_id,
AVG(Age) AS Average_Age
FROM Employees
WHERE Gender = 'F'
GROUP BY Location_id
HAVING AVG(Age) < 30;

```

OUTPUT:



Joins:

1. INNER JOIN:

- List employee names, their designations, and department names where employees are assigned to a department.

- ✓ For an **INNER JOIN**, connect the Employees table with the Departments table using department_id:

Syntax:

SELECT

e.employee_name,
e.designation,
d.department_name

FROM Employees e

INNER JOIN Departments d

ON e.department_id = d.department_id;

OUTPUT:

The screenshot shows the MySQL Workbench interface. The SQL Editor contains the following query:

```
SELECT
  e.employee_name,
  e.designation,
  d.department_name
FROM Employees e
INNER JOIN Departments d
ON e.department_id = d.department_id;
```

The Results window displays the output of the query in a table format:

employee_name	designation	department_name
Aarav Nair	Software Engineer	Software Development
Anushka Singh	Marketing Manager	Marketing
Saamir Patel	Marketing Analyst	Marketing
Anika Paul	Public Relations Specialist	Marketing
Vhaan Singh	Data Analyst	Data Science
Akash Kumar	Product Designer	Product Development

The Output window shows the execution log with the following entries:

#	Time	Action	Message	Duration / Fetch
45	23:46:54	SELECT department_id, COUNT(*) AS Employee_Count FROM Employees G...	7 row(s) returned	0.031 sec / 0.000 sec
46	23:49:50	SELECT Location_id, AVG(Age) AS Average_Age FROM Employees WHER...	3 row(s) returned	0.016 sec / 0.000 sec
47	23:52:57	SELECT e.employee_name, e.designation, d.department_name FROM E...	30 row(s) returned	0.000 sec / 0.000 sec

2. LEFT JOIN:

- List all departments along with the total number of employees in each department, including departments with no employees.

- ✓ For a **LEFT JOIN**, start with the Departments table so that **all departments** are included, even when they have no employees.

Syntax:

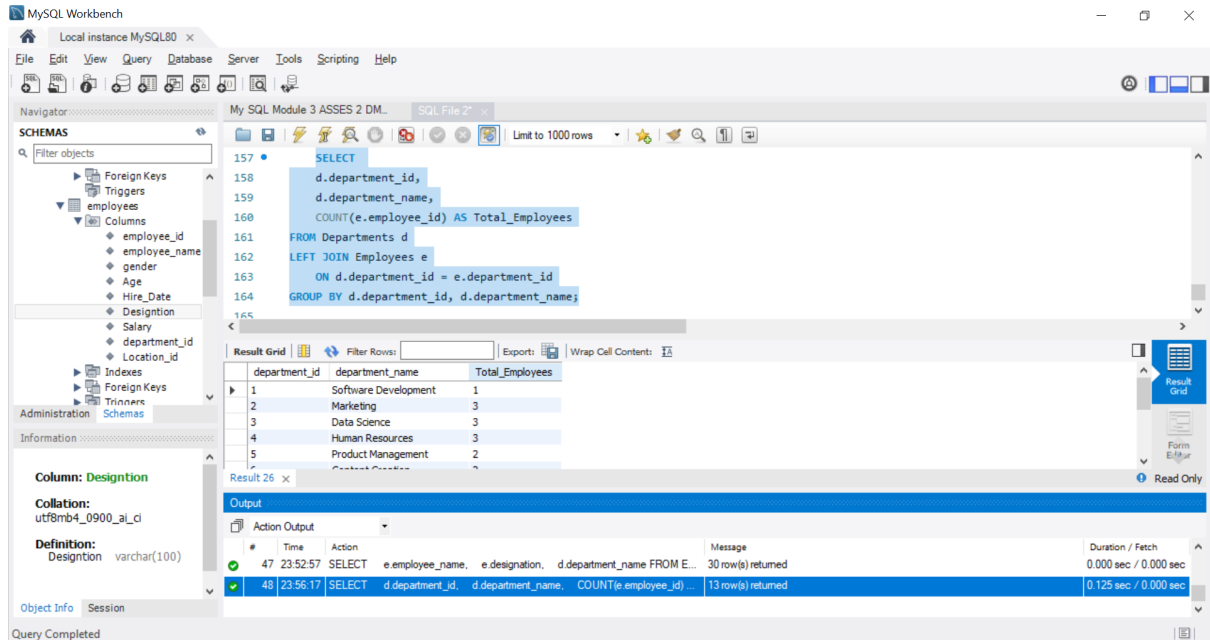
SELECT

```

        d.department_id,
        d.department_name,
COUNT(e.employee_id) AS Total_Employees
FROM Departments d
LEFT JOIN Employees e
ON d.department_id = e.department_id
GROUP BY d.department_id, d.department_name;

```

OUTPUT:



3. RIGHT JOIN:

- Display all locations along with the names of employees assigned to each location.

If no employees are assigned to a location, display NULL for employee name.

- ✓ For a **RIGHT JOIN**, put the Employees table on the left and the location table on the right. This ensures **all locations** are displayed, even if no employees are assigned.

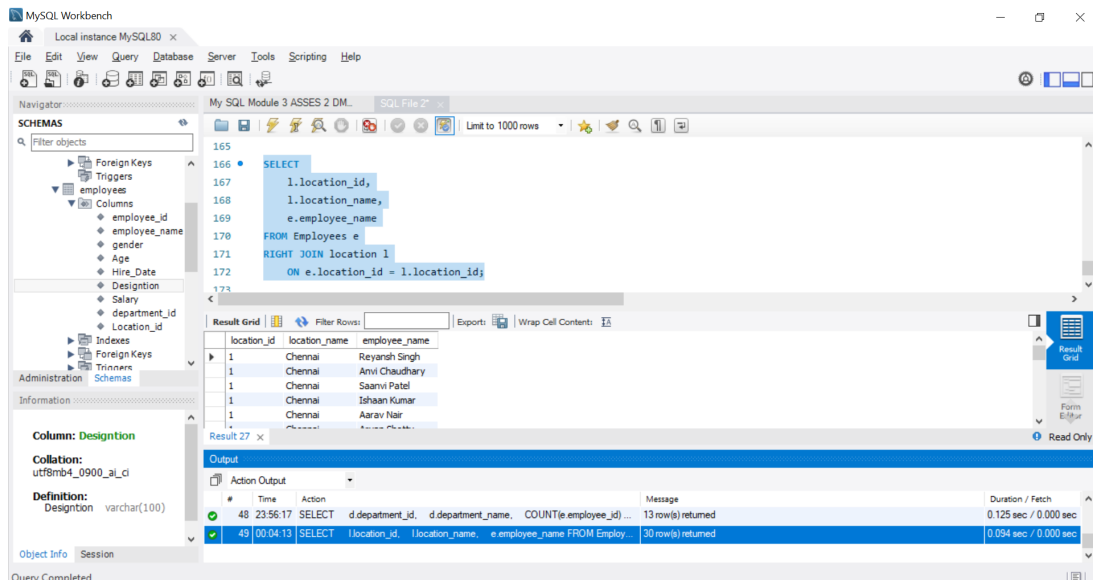
Syntax:

```

SELECT
    l.location_id,
    l.location_name,
    e.employee_name
FROM Employees e
RIGHT JOIN location l
ON e.location_id = l.location_id;

```

OUTPUT:



4. CROSS JOIN

- Show all possible combinations of departments and locations.

All possible combinations of departments and locations, use a **CROSS JOIN**:

Syntax:

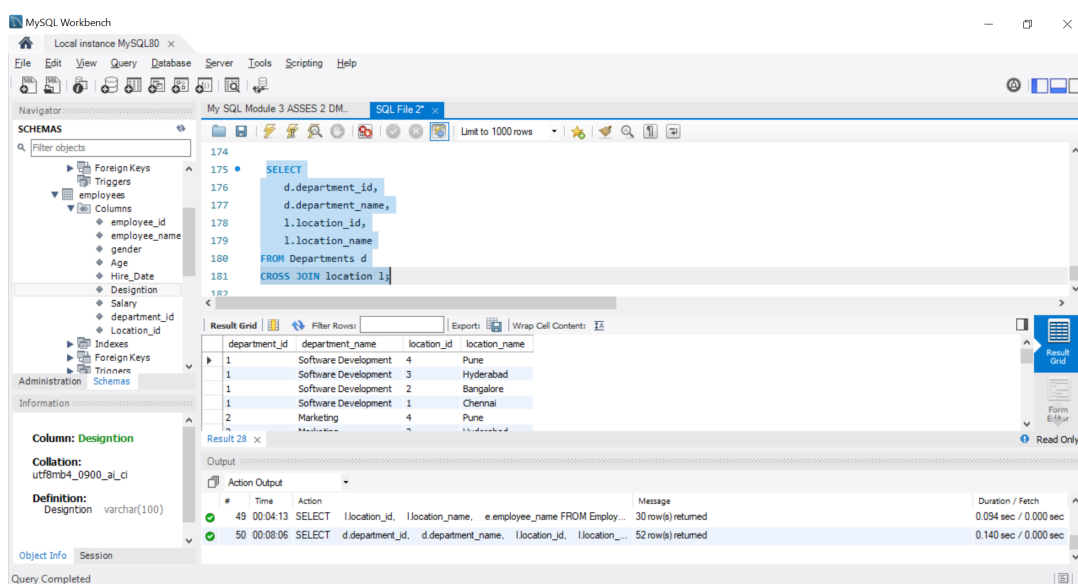
SELECT

 d.department_id,
 d.department_name,
 l.location_id,
 l.location_name

FROM Departments d

CROSS JOIN location l;

OUTPUT:



5. SELF JOIN:

- Show pairs of employees working in the same department, excluding self-pairs.
 - ✓ For a **SELF JOIN**, join the Employees table to itself using two different aliases:

Syntax:

SELECT

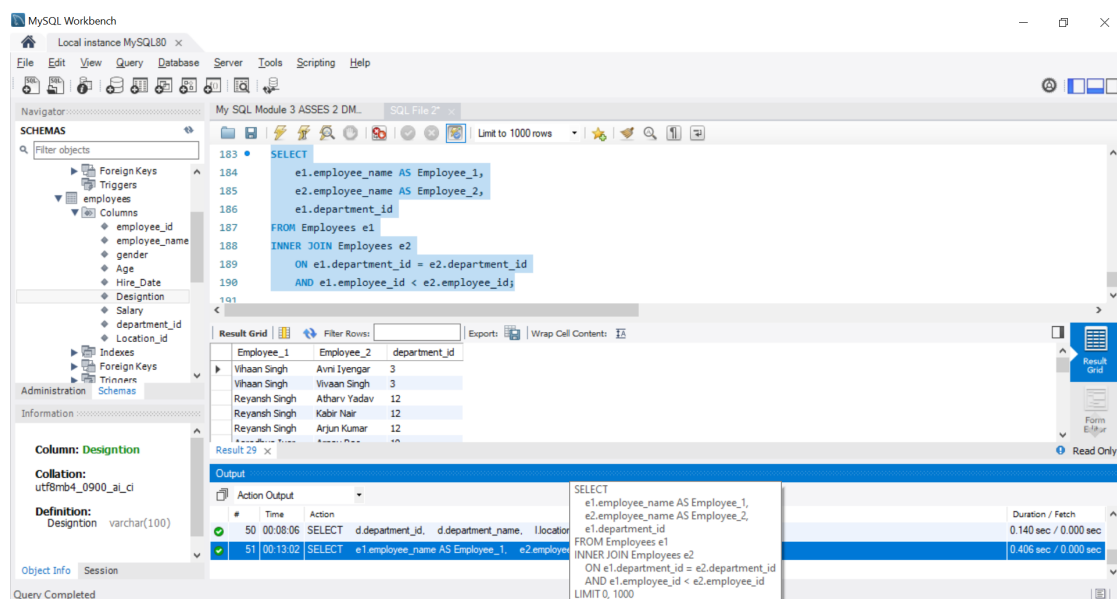
```
e1.employee_name AS Employee_1,  
e2.employee_name AS Employee_2,  
e1.department_id
```

FROM Employees e1

INNER JOIN Employees e2

```
ON e1.department_id = e2.department_id  
AND e1.employee_id < e2.employee_id;
```

OUTPUT:



The screenshot shows the MySQL Workbench interface. The query editor contains the following SQL query:

```
SELECT  
  e1.employee_name AS Employee_1,  
  e2.employee_name AS Employee_2,  
  e1.department_id  
FROM Employees e1  
INNER JOIN Employees e2  
  ON e1.department_id = e2.department_id  
  AND e1.employee_id < e2.employee_id;
```

The result grid displays the following data:

Employee_1	Employee_2	department_id
Vihaan Singh	Avni Iyengar	3
Vihaan Singh	Vivaan Singh	3
Reyansh Singh	Atharv Yadav	12
Reyansh Singh	Kabir Nair	12
Reyansh Singh	Arjun Kumar	12
Reyansh Singh	Karan Singh	12

The output pane shows the query execution details:

#	Time	Action	Duration / Fetch
50	00:08:06	SELECT d.department_id, d.department_name, l.location	0.140 sec / 0.000 sec
51	00:13:02	SELECT e1.employee_name AS Employee_1, e2.employee_name AS Employee_2, e1.department_id	0.406 sec / 0.000 sec

Windows function

- Write a window function query to rank employees by salary using rank().
 - ✓ Use the **RANK()** window function to rank employees based on their salary:

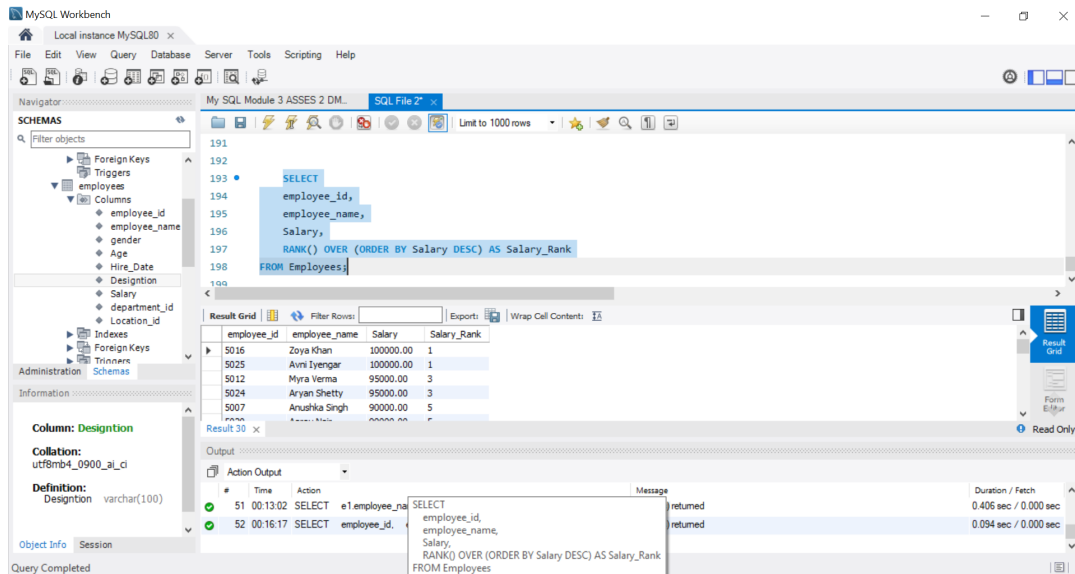
Syntax:

SELECT

```
employee_id,  
employee_name,  
Salary,  
RANK() OVER (ORDER BY Salary DESC) AS Salary_Rank
```

FROM Employees;

OUTPUT:



- Write a window function query to rank employees by salary within each department using **DENSE_RANK()**
 - ✓ Use **DENSE_RANK()** with **PARTITION BY department_id** to rank employees within each department:

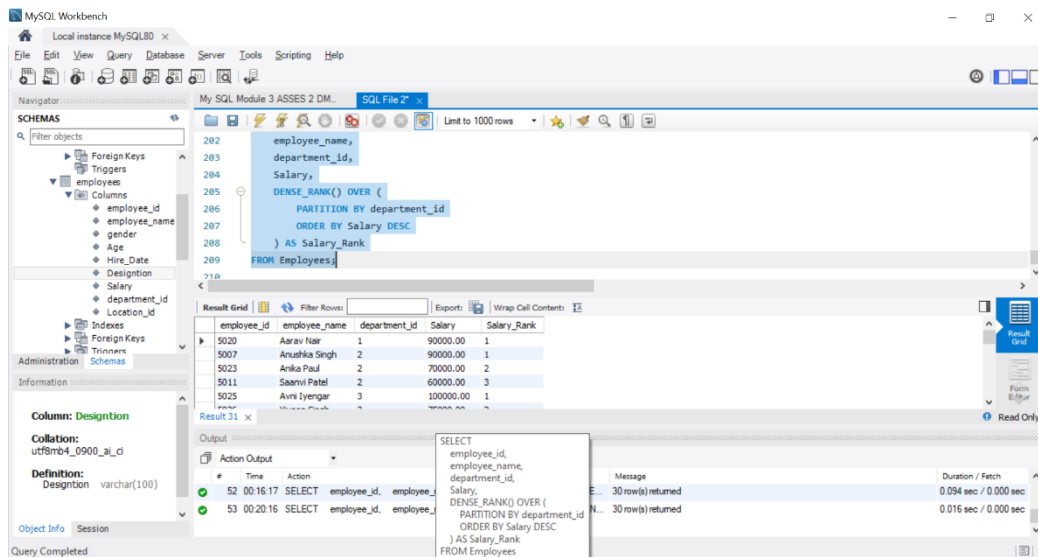
Syntax:

SELECT

 employee_id,
 employee_name,
 department_id,
 Salary,

DENSE_RANK() OVER (
 PARTITION BY department_id
 ORDER BY Salary DESC
) AS Salary_Rank

FROM Employees;



OUTPUT:

- Write a window function query, Running total salary by department

To calculate a **running total of salary within each department**, use **SUM()** as a window function with **PARTITION BY**

Syntax:

```

SELECT employee_id,
       employee_name,
       department_id,
       Salary,
       SUM(Salary) OVER (
         PARTITION BY department_id
         ORDER BY employee_id )
AS Running_Total_Salary
FROM Employees;

```

OUTPUT:

The screenshot displays the MySQL Workbench interface. The SQL Editor at the top contains a query that uses a window function to calculate a running total salary by department. The query is as follows:

```
SELECT
  employee_name,
  department_id,
  Salary,
  SUM(Salary) OVER (
    PARTITION BY department_id
    ORDER BY employee_id
  ) AS Running_Total_Salary
FROM Employees;
```

Below the editor, the 'Result Grid' shows the output of the query. It includes columns for employee_id, employee_name, department_id, Salary, and Running_Total_Salary. The data is sorted by department_id and then by employee_id.

employee_id	employee_name	department_id	Salary	Running_Total_Salary
5020	Aarav Nair	1	90000.00	90000.00
5007	Arushika Singh	2	90000.00	90000.00
5011	Saarnvi Patel	2	60000.00	150000.00
5023	Anika Paul	2	70000.00	220000.00
5001	Vhaan Singh	3	60000.00	60000.00

The 'Output' pane at the bottom shows the execution details of the query, including the time taken and the number of rows returned.

Conclusion

This DML assignment provided practical experience in managing and analyzing employee database records using MySQL. Various SQL commands and techniques were used to insert, retrieve, update, and organize employee data effectively.

The assignment demonstrated the use of **INSERT, SELECT, UPDATE, WHERE, DISTINCT, ORDER BY, and LIMIT clauses**, along with aggregate functions such as **SUM(), MIN(), MAX(), and AVG()**. It also covered **GROUP BY and HAVING** for analyzing employee data based on departments, salaries, and locations.

Different types of joins, including **INNER JOIN, LEFT JOIN, RIGHT JOIN, CROSS JOIN, and SELF JOIN**, were used to retrieve related information from multiple tables. Window functions such as **RANK(), DENSE_RANK()**, and running totals were also implemented to perform advanced employee salary analysis.

Overall, this assignment strengthened my understanding of **DML operations and SQL data analysis techniques**. It demonstrated how MySQL can be used to efficiently retrieve, modify, summarize, and analyse employee information while producing meaningful insights from structured data.